Machine Intelligence: Search, Planning & Theory Reference

Lectures 2–6: State Space, Heuristics, FDR, PDB, POR

Part	Topic	Lecture
A	Definitions & Key Concepts (State Space, Nodes, Costs)	L2
B	Uninformed Search (DFS, BFS, UCS, IDS)	L2
C	Informed Search (Heuristics, GBFS, A*, Hill Climbing)	L3
D	Classical Planning & FDR (Delete Relaxation, FF)	L4
E	Abstractions & Pattern Databases (PDB, Orthogonality)	L5
F	Partial Order Reduction (Action Dependencies, S3)	L6

Contents

A.1 State Space Fundamentals **5**

A.2 Cost Notation **5**

A.3 Search Node Structure **6**

A.4 Algorithm Evaluation Criteria **6**

A.5 Uninformed Search Algorithms **6**

A.6 Depth-First Search (DFS) **6**

A.7 Breadth-First Search (BFS) **7**

A.8 Uniform-Cost Search (UCS) **7**

A.9 Iterative Deepening Search (IDS) **8**

A.10 Greedy Best-First Search (GBFS) **8**

A.11 A* Search Algorithm **9**

B.12 State Space Drawing Convention **11**

B.13 UCS Table Format **11**

B.14 DFS Iteration Table Format **11**

B.15 BFS Iteration Table Format **12**

B.16 IDS Table Format **12**

B.17 GBFS Iteration Table Format **13**

B.18 A* Iteration Table Format **13**

B.19 BFS vs DFS Expansion Order **14**

B.20 Blackbox API Functions **14**

B.21 Algorithm Selection Guide **14**

C.22 Heuristic Functions **15**

C.23 Heuristic Properties (CRITICAL) **15**

C.24 Common Heuristics **15**

C.25 Local Search: Hill Climbing **16**

D.26 FDR: Finite Domain Representation **17**

D.27 Action Semantics **17**

D.28 Delete Relaxation **17**

D.29 Delete-Relaxation Heuristics **18**

D.30 SimpleDeleteRelaxationHeur Algorithm **18**

D.31 FF Heuristic (Fast Forward) **18**

D.32 Planning Complexity **19**

E.33 Abstraction Fundamentals **20**

E.34 Pattern Database (PDB) Heuristics **20**

E.35 Orthogonal Patterns (CRITICAL) **20**

E.36 Cliques Heuristic **21**

E.37 Pathological Abstractions **21**

F.38 Motivation: Why POR? **22**

F.39 Action Dependencies (CRITICAL) **22**

F.40 Strong Stubborn Sets (S3) **22**

F.41 S3 Admissibility **23**

F.42 Quick Reference **23**

QUICK REFERENCE TABLES

Search Algorithm Comparison

Algorithm	Complete?	Optimal?	Time	Space	Data Structure
UNINFORMED SEARCH					
DFS	**NO**	**NO**	$O(b^m)$	$O(bm)$	Stack (LIFO)
BFS	YES	YES*	$O(b^d)$	$O(b^d)$	Queue (FIFO)
IDS	YES	YES*	$O(b^d)$	$O(bd)$	Stack + depth limit
UCS	YES	YES	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$	$O(\cdot)$	Priority Q by $g(n)$
INFORMED SEARCH					
GBFS	YES†	**NO**	$O(b^m)$	$O(b^m)$	Priority Q by $h(n)$
A*	YES	YES**	$O(b^d)$	$O(b^d)$	Priority Q by $f(n)$
Hill Climbing	**NO**	**NO**	Variable	$O(b)$	None (greedy)

*Unit costs only **Requires admissible h †With duplicate elimination

Key: b=branching factor, d=goal depth, m=max depth, ϵ=min action cost

Heuristics Comparison

Heuristic	Adm?	Time	Plain English
h^*	YES	Unknown	The TRUE cost (what we want but can't compute)
h^+	YES	NP-hard	Best relaxed cost (ignore delete effects)
$h^{\max}$	YES	Poly	Cost of hardest single goal fact (conservative)
h^{add}	**NO**	Poly	Sum costs of all goal facts (double-counts!)
h^{FF}	**NO**	Poly	Greedy plan extraction from RPG layers
h^P (PDB)	YES	$O(1)$	Precomputed lookup for subset of variables
h^{clique}	YES	$O(1)$	Best sum of orthogonal PDBs (see below)

Relationship: $h^{\max} \leq h^+ \leq h^*$ (all admissible ones never overestimate)

h^{clique} — Combining Multiple PDBs

Problem: One small PDB may be weak. Can we combine multiple?
Naive SUM: Adding any two PDBs can **overestimate** (double-count actions)
Solution: Only SUM **orthogonal** patterns (no action affects both)
Formula: $h^{\text{clique}}(s) = \max_{\text{clique } D} \sum_{P \in D} h^P(s)$
Plain English: Find groups of patterns that don't interfere. Sum within each group. Take the best (max) result.

Action Dependencies (POR/L6)

Relationship	Plain English Meaning
Enabling	Action a_1 makes action a_2 possible (unlocks it)
Disabling	Action a_1 makes action a_2 impossible (locks it)
Conflict	Actions give different results depending on order
Interfering	Actions either conflict or one disables the other
Commutative	Actions are completely independent (the good case!)

Key: Interfering = Conflict OR Disabling Commutative = NOT Interfering AND NOT Enabling

Planning/Analysis Algorithms

Algorithm	Purpose	Output	Adm?
SimpleDeleteHeur	Delete relax (layers)	h-value	Approx
FF Forward	Build RPG layers	Fact levels	—
FF Backward	Extract relaxed plan	h^{FF}	**NO**
NES (Goal)	Find $\mathcal{A}_{s \to^* G}$	Goal-needed actions	—
NES (Action)	Find $\mathcal{A}_{s \to^* a}$	Enablers of a	—
S3	Build stubborn set	Pruned $\rho(s)$	YES
PDB Construction	Backward Dijkstra	h^P table	YES

Heuristic Property Implications

Consistent $\Rightarrow$ **Admissible** $\Rightarrow$ **Safe**

(But: Safe $\not\Rightarrow$ Admissible $\not\Rightarrow$ Consistent)

Property	Definition	Guarantees
Safe	$h(s) = \infty \Rightarrow h^*(s) = \infty$	No false dead-ends
Admissible	$h(s) \leq h^*(s)$ for all s	A* optimality
Consistent	$h(s) \leq \text{cost}(a) + h(s')$	No reopening needed

PART A: Definitions & Key Concepts

A.1 State Space Fundamentals

State Space Definition

A **state space** is a tuple:

$$\Theta = (S, A, \text{cost}, T, I, S_G)$$

- S = finite set of **states**
- A = finite set of **actions**
- $\text{cost} : A \to \mathbb{N}_0$ = **cost function** (non-negative)
- $T \subseteq S \times A \times S$ = **transition relation** (deterministic)
- $I \in S$ = **initial state** (exactly one)
- $S_G \subseteq S$ = **goal states** (can be multiple)

Transition notation: $(s, a, s') \in T$ written as $s \xrightarrow{a} s'$

Action applicability: Action a is applicable in s iff $\exists s'$ such that $(s, a, s') \in T$

State Relationships

Successor	s' is successor of s if $s \xrightarrow{a} s'$ for some a
Predecessor	s is predecessor of s' if $s \xrightarrow{a} s'$ for some a
Reachable from s	s' reachable from s if $\exists$ path $s = s_0 \to s_1 \to \cdots \to s_n = s'$
Reachable	s is reachable if reachable from I
Solvable	s is solvable if some $g \in S_G$ is reachable from s
Dead-end	s is dead-end if no goal reachable from s

Path Types — Exact Format

A **path** is a sequence of transitions: $\tau = ((s_0, a_1, s_1), (s_1, a_2, s_2), \ldots, (s_{n-1}, a_n, s_n))$

Three representations:

- **Transition-path:** $((B, d, J), (J, e, E), (E, i, F))$ or $B \xrightarrow{d} J \xrightarrow{e} E \xrightarrow{i} F$
- **Action-path:** (d, e, i)
- **State-path:** (B, J, E, F)

Path cost: $\text{cost}(\tau) = \text{cost}(\pi) = \sum_{i=1}^{n} \text{cost}(a_i)$

Empty path: Valid with $n = 0$. Every state reachable from itself with cost 0.

Solutions

- **Solution for state** s: Path from s to some $g \in S_G$
- **Solution for** Θ: Solution for initial state I
- **Optimal solution**: Solution with **minimal cost**
- **Solvable**: Θ is solvable iff a solution exists
- **Unit cost**: All actions have cost 1

A.2 Cost Notation

Important Cost Values

$g(n)$	Path cost from I to node n (actual cost incurred)
$g^*(s)$	**Optimal** cost to reach state s from I
g^*	Cost of optimal solution (from I to any goal)
$h(s)$	Heuristic estimate of cost from s to goal
$h^*(s)$	**True** optimal cost from s to nearest goal
$f(n)$	$g(n) + h(n)$ — estimated total cost through n

g-value update: $g(n') = g(n) + \text{cost}(a)$ where $n \xrightarrow{a} n'$

A.3 Search Node Structure

Search Node vs State

A **search node** n contains:

- `n.State`: The state this node represents
- `n.Pred`: Pointer to parent node (or `null`)
- `n.Action`: Action that led to this node (or `null`)
- $g(n)$: Cost of path from I to this node

Key distinction: Same state can appear in multiple nodes (different paths)!

Search Terminology

Node expansion	Generate all successors of a node
Open list (frontier)	Nodes waiting to be expanded
Closed list (explored)	States already expanded (graph search only)
Tree search	No duplicate elimination, no closed list
Graph search	With duplicate elimination, uses closed list

A.4 Algorithm Evaluation Criteria

Four Properties

Complete	Guaranteed to find a solution if one exists
Optimal	Returned solution is guaranteed minimal cost
Time complexity	Number of nodes expanded (typically $O(b^d)$ or $O(b^m)$)
Space complexity	Maximum nodes stored simultaneously

Parameters:

- b = branching factor (average successors per state)
- d = goal depth (minimum depth of any goal)
- m = maximum depth of state space
- ϵ = minimum positive action cost

A.5 Uninformed Search Algorithms

Algorithm Comparison

Algorithm	Data Struct.	Complete?	Optimal?	Time	Space
DFS	Stack (LIFO)	No*/Yes**	No	$O(b^m)$	$O(bm)/O(b^m)$
BFS	Queue (FIFO)	Yes	Unit cost	$O(b^d)$	$O(b^d)$
UCS	Priority (by g)	Yes	**Yes**	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$
IDS	Stack + depth	Yes	Unit cost	$O(b^d)$	$O(bd)$

*Tree search **Graph search

Critical Distinctions

- **UCS is ALWAYS optimal** (any non-negative costs)
- **BFS and IDS optimal ONLY for unit cost**
- **IDS has best space:** $O(bd)$ — linear!
- **DFS tree search NOT complete** (can cycle forever)
- **UCS ≡ Dijkstra's algorithm** on state space graph

A.6 Depth-First Search (DFS)

DFS Strategy

Core idea: Go as deep as possible, then backtrack when stuck

Data structure: LIFO queue (stack) — Last In, First Out

Behavior: Explores one branch completely before trying another

DFS Pseudocode (Graph Search)

```
n ← InitNode() {Create node for initial state}
Q ← empty stack; Push(Q, n) {STACK = LIFO = go DEEP}
closed ← ∅ {Remember what we've expanded}
while Q not empty do
  n ← Pop(Q) {Take LAST added (most recent)}
  if GoalTest(n.State) then
    return ExtractSolution(n) {Follow parent pointers back}
  end if
  closed ← closed ∪ {n.State} {Mark as visited}
  for each a ∈ Actions(n.State) do
    {Try all applicable actions}
    n′ ← SuccNode(n, a) {Make child node}
    if n′.State ∉ closed then
      {Only if not seen before}
      Push(Q, n′) {Add to stack}
    end if
  end for
end while
return "unsolvable" {Exhausted all options}
```

Plain English: Stack means we dive deep before backtracking. No g-values $\Rightarrow$ NOT optimal.

A.7 Breadth-First Search (BFS)

BFS Strategy

Core idea: Explore all nodes at depth d before any at depth $d+1$
Data structure: FIFO queue — First In, First Out
Behavior: Level-by-level exploration (shallowest first)

BFS Pseudocode

```
n ← InitNode() {Create node for initial state}
Q ← empty queue; Enqueue(Q, n) {QUEUE = FIFO = level by level}
while Q not empty do
  n ← Dequeue(Q) {Take OLDEST (first added)}
  if GoalTest(n.State) then
    return ExtractSolution(n) {Follow parent pointers back}
  end if
  for each a ∈ Actions(n.State) do
    {Try all applicable actions}
    n′ ← SuccNode(n, a) {Make child node}
    Enqueue(Q, n′) {Add to back (will wait its turn)}
  end for
end while
return "unsolvable" {Exhausted all options}
```

Plain English: Queue means we explore ALL nodes at depth d before ANY at depth d+1. Shallowest goal found first $\Rightarrow$ optimal ONLY if all actions cost the same!

Why BFS Optimal for Unit Cost Only

If all actions cost 1: path cost = depth $\Rightarrow$ shallowest = cheapest
If costs differ: deeper node might have lower total cost than shallower node!

A.8 Uniform-Cost Search (UCS)

UCS Strategy

Core idea: Always expand node with **lowest** $g(n)$
Data structure: Priority queue ordered by g-value
Key insight: When we expand a node, we've found optimal path to it!

UCS Pseudocode (Graph Search)

```
n ← InitNode(); g(n) ← 0 {Start node costs 0 to reach}
open ← priority queue by g; Insert n {Expand CHEAPEST so far}
closed ← ∅ {Remember what we've fully explored}
while open not empty do
  n ← Pop(open) {Get node with LOWEST g(n)}
  if GoalTest(n.State) then
    {First time we pop goal = optimal!}
    return ExtractSolution(n)
  end if
  closed ← closed ∪ {n.State} {This state is done}
  for each a ∈ Actions(n.State) do
    n′ ← SuccNode(n, a) {g(n′) = g(n) + cost(a)}
    if n′.State ∉ closed and n′.State ∉ States(open) then
      Insert n′ into open {New state discovered}
    else if ∃n″ ∈ open with n″.State = n′.State and g(n′) < g(n″) then
      Replace n″ with n′ in open {Found CHEAPER path!}
    end if
  end for
end while
return "unsolvable"
```

Plain English: Priority queue by g = always expand the node we've reached most cheaply. When we first pop a goal, that's guaranteed to be the optimal path. This is basically Dijkstra's algorithm!

A.9 Iterative Deepening Search (IDS)

IDS Strategy

Core idea: Run DFS with depth limit 0, 1, 2, ... until goal found
Purpose: BFS completeness + optimality (unit cost) with DFS memory
Overhead: Only ~11% more work than BFS (most work at deepest level)

IDS Pseudocode

```
function IterativeDeepening
n ← InitNode()
for depth = 0 to ∞ do
  result ← DFSLimited(n, depth)
  if result ≠ "cutoff" then
    return result
  end if
end for
```

DFSLimited(n, depth): Returns solution, "cutoff", or "unsolvable"

- If goal: return solution
- If depth = 0: return "cutoff"
- Recursively call on successors with depth−1
- Return "cutoff" if any child returned cutoff, else "unsolvable"

A.10 Greedy Best-First Search (GBFS)

GBFS Strategy

Core idea: Always expand node with **lowest** $h(n)$
Data structure: Priority queue ordered by h-value (ascending)
Key difference from UCS: Uses $h(n)$ only, ignores $g(n)$

GBFS Pseudocode

```
n ← InitNode() {Create node for initial state}
open ← priority queue by h; Insert n {Expand closest-LOOKING}
closed ← ∅ {Remember what we've expanded}
while open not empty do
  n ← Pop(open) {Get node with LOWEST h(n)}
  if GoalTest(n.State) then
    {Found goal (maybe not optimal!)}
    return ExtractSolution(n)
  end if
  closed ← closed ∪ {n.State}
  for each a ∈ Actions(n.State) do
    n′ ← SuccNode(n, a)
    if n′.State ∉ closed and n′.State ∉ States(open) then
      Insert n′ into open {NOTE: No cheaper-path check!}
    end if
  end for
end while
return "unsolvable"
```

Plain English: Greedy = always go toward what LOOKS closest to goal (lowest h). Ignores actual cost paid (g). Fast but can find expensive solutions!

GBFS Properties

Complete?	Yes (with duplicate elimination)
Optimal?	**NO** — follows h greedily
Time:	$O(b^m)$
Space:	$O(b^m)$

No cheaper path logic! If seen state before, GBFS ignores new path.

A.11 A* Search Algorithm

A* Evaluation Function

A* combines both costs:

$$f(n) = g(n) + h(n)$$

- $g(n)$ = actual cost from start to n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost through n

A* Pseudocode

```
n ← InitNode(); g(n) ← 0; f(n) ← h(n.State) {f = g + h}
open ← priority queue by f; Insert n {Expand most PROMISING}
closed ← ∅
while open not empty do
  n ← Pop(open) {Lowest f(n) = g(n) + h(n)}
  if GoalTest(n.State) then
    {Optimal if h admissible!}
    return ExtractSolution(n)
  end if
  closed ← closed ∪ {n.State}
  for each a ∈ Actions(n.State) do
    n′ ← SuccNode(n, a) {Creates child, sets g(n′)}
    if n′.State ∉ closed and n′.State ∉ States(open) then
      Insert n′ into open {New state}
    else if ∃n″ ∈ open with same state and g(n′) < g(n″) then
      Replace n″ with n′ {Cheaper path}
    else if ∃n″.State ∈ closed and g(n′) < g(n″) then
      Remove from closed; Insert n′ into open {Reopening!}
    end if
  end for
end while
return "unsolvable"
```

Plain English: $f = g + h$ balances what we've spent (g) with what we expect to spend (h). Combines UCS's optimality with heuristic guidance. **With admissible h: always finds optimal solution!**

A* Properties

Property	Value	Condition
Complete?	Yes	Safe h, finite b, costs $> \epsilon$
Optimal?	**Yes**	Admissible h
Time	$O(b^d)$	Exponential
Space	$O(b^d)$	Stores all nodes

With consistent h: No reopening needed (line for reopening never executes)

A* Optimality Proof Intuition

When A* expands goal G:

1. $f(G) = g(G) + h(G) = g(G) + 0 = g(G)$
2. All remaining nodes in open have $f(n) \geq f(G)$
3. If cheaper goal existed, some node on optimal path would have $f < f(G)$
4. But A* expanded G first $\Rightarrow$ no such node exists

PART B: Problem-Solving Patterns & Table Formats

B.12 State Space Drawing Convention

Drawing Elements

- **States:** Circles with state name inside
- **Initial state I:** Incoming arrow with no source
- **Goal states S_G:** Double circles
- **Transitions:** Labeled directed edges (action names on edges)
- **Costs:** Numbers on edges (if not unit cost)

Finding Reachable and Solvable States

Reachable states: BFS/DFS **forward** from I
Solvable states: BFS/DFS **backward** from S_G (reverse all edges)
Dead-ends: Reachable but not solvable

B.13 UCS Table Format

UCS Iteration Table

Step	State	g-value	Closed
1	I	0	$\{I\}$
2	next lowest g	g	$\{I, \ldots\}$
3	...	...	...

Key points:

- **Priority queue:** Always pop node with lowest g-value
- **Closed set:** Add state to closed AFTER popping
- **Stop when:** Goal state popped from open list
- **Tie-breaking:** Alphabetical/lexicographical order

UCS Example (Taxi Problem)

Step	t	p_1	p_2	g	Closed
1	A	C	E	0	$\{(A, C, E)\}$
2	B	C	E	1	$\{.., (B, C, E)\}$
3	D	C	E	2	$\{.., (D, C, E)\}$
4	C	C	E	3	$\{.., (C, C, E)\}$
5	E	C	E	3	$\{.., (E, C, E)\}$
6	C	t	E	4	$\{.., (C, t, E)\}$
7	E	C	t	4	$\{.., (E, C, t)\}$
8	F	C	E	4	$\{.., (F, C, E)\}$
9	F	C	t	5	$\{.., (F, C, t)\}$
10	F	C	F	6	GOAL

State encoding: (t, p_1, p_2) where t = taxi location, p_i = passenger location (t = in taxi)

B.14 DFS Iteration Table Format

DFS Iteration Table

Step	Node Popped	Children Pushed to Stack	Closed
0	—	S_0 (initial state)	$\emptyset$
1	S_0	$S_1, S_2, \ldots$ (successors)	$\{S_0\}$
2	S_k (last pushed)	children of S_k	$\{S_0, S_k\}$
...	...	...	...

Key points:

- **LIFO order:** Last child pushed = first popped next step
- **Closed set:** Add node to closed AFTER popping (when expanded)
- **Stop when:** Goal state popped OR stack empty

DFS Example Trace

Step	Node Popped	Children Pushed	Closed
0	—	S_0	$\emptyset$
1	S_0	$S_1, S_2, S_3, S_4, S_5, S_6$	$\{S_0\}$
2	S_6	$S_{11}, S_{15}, S_{18}, S_{20}, S_{21}$	$\{S_0, S_6\}$
3	S_{21}	$S_{26}, S_{27}, S_{28}, S_{29}$	$\{S_0, S_6, S_{21}\}$
4	S_{29}	S_{32}, S_{33}, S_{34}	$\{.., S_{29}\}$
5	S_{34}	S_{37}, S_{38}	$\{.., S_{34}\}$
6	S_{38}	S_{40}	$\{.., S_{38}\}$
7	S_{40}	none (goal)	$\{.., S_{40}\}$

Note: DFS explores deeply along one branch before backtracking.

B.15 BFS Iteration Table Format

BFS Iteration Table

Step	Node Dequeued	Children Added to Queue	Closed
0	—	S_0 (initial state)	$\emptyset$
1	S_0	$S_1, S_2, \ldots$ (successors)	$\{S_0\}$
2	S_1 (first in queue)	children of S_1	$\{S_0, S_1\}$
...	...	...	...

Key points:
- **FIFO order:** First child added = first dequeued (level-by-level)
- **Closed set:** Add node to closed AFTER dequeuing (when expanded)
- **Stop when:** Goal state dequeued OR queue empty

BFS Example Trace

Step	Node Dequeued	Children Added	Closed
0	—	S_0	$\emptyset$
1	S_0	$S_1, S_2, S_3, S_4, S_5, S_6$	$\{S_0\}$
2	S_1	$S_7, S_8, S_9, S_{10}, S_{11}$	$\{S_0, S_1\}$
3	S_2	$S_{12}, S_{13}, S_{14}, S_{15}$	$\{.., S_2\}$
4	S_3	S_{16}, S_{17}, S_{18}	$\{.., S_3\}$
5	S_4	S_{19}, S_{20}	$\{.., S_4\}$
6	S_5	S_{21}	$\{.., S_5\}$
7	S_6	none (all in closed)	$\{.., S_6\}$
8	S_7	$S_{22}, S_{23}, S_{24}, S_{25}$	$\{.., S_7\}$

Note: BFS explores all nodes at depth d before any at depth $d+1$.

B.16 IDS Table Format

IDS Iteration Table

depth limit	step	state
0	1	I
1	1	I
1	2	child_1
1	3	child_2
2	1	I
2	2	child_1
2	3	grandchild_{11}
...	...	...

Each depth limit: Complete DFS up to that depth, then restart
Order: DFS order within each depth limit (last child first)

B.17 GBFS Iteration Table Format

GBFS Table Format

Step	State	h	Closed
1	I	$h(I)$	$\{I\}$
2	next lowest h	h	$\{I, \ldots\}$

Key points:
- **Priority queue:** Always pop node with lowest $h(n)$ (heuristic only!)
- **Ignores g-value:** Path cost does NOT matter — pure greedy
- **Closed set:** Add state to closed AFTER popping
- **No cheaper path check:** Unlike UCS/A*, GBFS doesn't update paths
- **Tie-breaking:** Alphabetical/lexicographical order
- **Stop when:** Goal state popped from open

GBFS Example Trace

Step	State	h	Closed
1	A	5	$\{A\}$
2	B	3	$\{A, B\}$
3	D	2	$\{A, B, D\}$
4	G	0	GOAL

Note: GBFS greedily follows lowest h. May find suboptimal solution!

B.18 A* Iteration Table Format

A* Table Format

Step	State	g	h	$f = g + h$	Closed
1	I	0	$h(I)$	$h(I)$	$\{I\}$
2	next lowest f	g	h	f	$\{I, \ldots\}$

Key points:
- **Priority queue:** Always pop node with lowest $f = g + h$
- **Closed set:** Add state to closed AFTER popping
- **Tie-breaking:** Alphabetical (or by lower h, then alphabetical)
- **Stop when:** Goal state popped from open

GBFS vs UCS vs A* — What Gets Expanded

Algorithm	Priority	Guarantees
UCS	$g(n)$ only	Optimal (ignores goal direction)
GBFS	$h(n)$ only	Fast but **NOT optimal**
A*	$g(n) + h(n)$	Optimal (with admissible h)

B.19 BFS vs DFS Expansion Order

Queue Operations

BFS (Queue - FIFO):

```
Open: [A] -> pop A, add children [B,C]
Open: [B,C] -> pop B, add children [D,E]
Open: [C,D,E] -> pop C...
(level by level)
```

DFS (Stack - LIFO):

```
Open: [A] -> pop A, add children [B,C]
Open: [C,B] -> pop C, add children [F,G]
Open: [G,F,B] -> pop G...
(deepest first)
```

B.20 Blackbox API Functions

State Space API

`InitialState()`	Returns initial state I
`GoalTest(s)`	Returns true iff $s \in S_G$
`Cost(a)`	Returns cost of action a
`Actions(s)`	Returns set of actions applicable in s
`Successor(s, a)`	Returns s' where $s \xrightarrow{a} s'$

Helper Functions

InitNode():
- n.State $\leftarrow$ InitialState()
- n.Pred $\leftarrow$ `null`; n.Action $\leftarrow$ `null`; $g(n) \leftarrow 0$

SuccNode(p, a):
- n.State $\leftarrow$ Successor(p.State, a)
- n.Pred $\leftarrow p$; n.Action $\leftarrow a$
- $g(n) \leftarrow g(p) + \text{Cost}(a)$

ExtractSolution(n): Follow Pred pointers backward, collect actions

B.21 Algorithm Selection Guide

Decision Tree

Need optimality?
- **Yes + non-unit costs:** Use **UCS**
- **Yes + unit costs + memory OK:** Use **BFS**
- **Yes + unit costs + limited memory:** Use **IDS**
- **No (any solution OK):** Use **DFS** (graph search)

Common Mistakes

1. Using BFS for non-unit costs — **NOT optimal!**
2. Forgetting duplicate elimination in DFS — can loop forever
3. Not updating g-values in UCS when finding cheaper path
4. Confusing open (candidates) vs closed (already expanded)

PART C: Informed Search (L3)

C.22 Heuristic Functions

Heuristic Function

A **heuristic function** is a mapping:

$$h : S \to \mathbb{N}_0 \cup \{\infty\}$$

where $h(s)$ estimates the cost from state s to the nearest goal.

Key notation:

- $h(s)$ = heuristic estimate for state s
- $h^*(s)$ = **perfect heuristic** (true optimal cost from s to goal)

C.23 Heuristic Properties (CRITICAL)

Three Key Properties

1. SAFE: $h(s) = \infty \Rightarrow h^*(s) = \infty$

- **Plain English:** If h says "impossible", it's truly impossible
- Only marks TRUE dead-ends as ∞ (never lies about reachability)

2. ADMISSIBLE: $h(s) \leq h^*(s)$ for all states s

- **Plain English:** h is always optimistic (never overestimates)
- Why it matters: A* won't skip a good path thinking it's bad

3. CONSISTENT (Monotonic): $h(s) \leq \text{cost}(a) + h(s')$ for all $s \xrightarrow{a} s'$

- **Plain English:** Estimate can't drop faster than actual cost
- Why it matters: f-values never decrease along a path $\Rightarrow$ no reopening needed

Implication Chain

Consistent $\Rightarrow$ Admissible $\Rightarrow$ Safe

But: Safe $\not\Rightarrow$ Admissible $\not\Rightarrow$ Consistent

Checking Properties — Table Method

Given graph $A \xrightarrow{1} B \xrightarrow{1} C$ (goal), $B \xrightarrow{1} D$ (dead-end):

h^*: $A = 2$, $B = 1$, $C = 0$, $D = \infty$

To check Safe: For each n where $h(n) = \infty$, verify $h^*(n) = \infty$

To check Admissible: For each n, verify $h(n) \leq h^*(n)$

To check Consistent: For each edge $n \to n'$ with cost c, verify $h(n) \leq c + h(n')$

C.24 Common Heuristics

Grid/Route Heuristics

4-directional (N, S, E, W): Manhattan distance

$$h = |x_1 - x_g| + |y_1 - y_g|$$

8-directional (diagonals): Chebyshev distance

$$h = \max(|x_1 - x_g|, |y_1 - y_g|)$$

Any-angle: Euclidean distance

$$h = \sqrt{(x_1 - x_g)^2 + (y_1 - y_g)^2}$$

Route planning: Straight-line distance (always admissible)

8-Puzzle Heuristics

h_1**:** Number of misplaced tiles

h_2**:** Manhattan distance (sum of tile distances)

Dominance: $h_2 \geq h_1$ for all states $\Rightarrow$ h_2 better

C.25 Local Search: Hill Climbing

Hill Climbing

Core idea: Start at current state, move to best neighbor
Strategy: $\arg\min_{s' \in \text{neighbors}} h(s')$
Termination: No neighbor is better than current state

Hill Climbing Pseudocode

```
current ← InitialState() {Start somewhere}
while true do
    {Keep going until stuck}
    neighbors ← all states reachable in one step from current
    next ← the neighbor with the LOWEST h value
    if h(next) ≥ h(current) then
        {No neighbor is better?}
        return current {Stuck! This is a local optimum}
    end if
    current ← next {Move to the better neighbor}
end while
```

Plain English:

1. Start at initial state
2. Look at all neighbors (states one action away)
3. Pick the neighbor with best (lowest) h value
4. If that neighbor is NOT better than current $\Rightarrow$ STOP (we're at a local optimum)
5. Otherwise, move there and repeat

Hill Climbing Limitations

Complete? **No** — stuck at local optima
Optimal? **No** — finds local, not global optimum
Space: $O(1)$ — only stores current state
Problems:

- Local maxima/minima
- Plateaus (flat regions)
- Ridges

Solution: Random-restart hill climbing (run multiple times)

PART D: Classical Planning & FDR (L4)

D.26 FDR: Finite Domain Representation

FDR Planning Task

A **planning task** is a tuple:

$$\Pi = (V, A, \text{cost}, I, G)$$

- V = finite set of **state variables**, each $v \in V$ has domain D_v
- A = finite set of **actions**, each $a = (\text{pre}_a, \text{eff}_a)$
- $\text{cost} : A \to \mathbb{N}_0$ = action cost function
- I = **initial state** (complete variable assignment)
- G = **goal** (partial variable assignment)

State space size: $|\Theta_\Pi| = \prod_{v \in V} |D_v|$

Facts and Partial States

Fact: (v, d) where $v \in V$ and $d \in D_v$. Written as $v{=}d$

Partial state: Set of facts, each variable appears **at most once**

Complete state: Each variable appears **exactly once**

D.27 Action Semantics

Action Application

Action $a = (\text{pre}_a, \text{eff}_a)$

Applicability: a applicable in state s iff $\text{pre}_a \subseteq s$

Transition:

$$s' = s[a] = (s \setminus \{(v, d) \in s \mid v \in \text{vars}(\text{eff}_a)\}) \cup \text{eff}_a$$

In words: Remove old values of affected variables, add new effect values

Action Application Example

State: $s = \{truck{=}A, pkg{=}C, fuel{=}full\}$

Action: $drive_AB$ with $\text{pre} = \{truck{=}A\}$, $\text{eff} = \{truck{=}B\}$

Result: $s' = \{truck{=}B, pkg{=}C, fuel{=}full\}$

D.28 Delete Relaxation

Delete-Relaxed Problem Π^+

Key idea: Once a fact becomes true, it STAYS true forever

State transition:

$$s^+[a] = s^+ \cup \text{eff}_a$$

(Just add effects, never remove anything!)

Plain English: Imagine you never lose anything you gain. Picked up a key? You have it forever. This makes planning EASIER (why it's called "relaxation").

Consequence: Variables can have multiple values simultaneously (weird but useful!)

The h^+ Heuristic

$$h^+(s) = \min_{\pi^+} \text{cost}(\pi^+)$$

where π^+ is a delete-relaxed plan from s to goal.

Properties:

- **Admissible:** $h^+(s) \leq h^*(s)$
- **NP-hard** to compute exactly

D.29 Delete-Relaxation Heuristics

Heuristic Comparison

Heuristic	Admissible?	Complexity	Informativeness
h^+	Yes	NP-hard	Optimal relaxed
$h^{\max}$	Yes	Polynomial	Conservative
h^{add}	**No**	Polynomial	More informed
h^{FF}	**No**	Polynomial	Very informed

Relationship: $h^{\max}(s) \leq h^+(s) \leq h^*(s)$

D.30 SimpleDeleteRelaxationHeur Algorithm

Algorithm (Layer-Based)

$F \leftarrow s$; $C \leftarrow 0$; $\mathcal{A} \leftarrow A$
while $G \not\subseteq F$ **do**
  $\mathcal{A}' \leftarrow \{a \in \mathcal{A} \mid \text{pre}_a \subseteq F\}$ ▷ Applicable actions
  $F' \leftarrow F \cup \bigcup_{a \in \mathcal{A}'} \text{eff}_a$
  if $F = F'$ **then**
    return ∞
  end if
  $C \leftarrow C + \min_{a \in \mathcal{A}'} \text{cost}(a)$
  $\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{A}'$; $F \leftarrow F'$
end while
return C

Delete Relaxation Table Format

Layer	Facts F_i	Goal?	Applicable Actions	Cost C
0	(initial state)	No	(list actions)	0
1	$F_0 \cup$ new effects	No	(list actions)	$C +$ min cost

D.31 FF Heuristic (Fast Forward)

FF Algorithm Overview

Phase 1: Forward RPG Construction

1. Build layers: $F_0 = s$
2. $F_{i+1} = F_i \cup \bigcup_{a \in A_i} \text{eff}_a$
3. Track when each fact first appears: level$[f]$
4. Stop when $G \subseteq F_i$

Phase 2: Backward Plan Extraction

1. $SG[\text{goal_layer}] = G$
2. For each layer j down to 1:
3. For each fact $f \in SG[j]$ with level$[f] = j$:
   - Choose action $a \in A_{j-1}$ with $f \in \text{eff}_a$
   - Add a to plan
   - Add pre_a to $SG[j-1]$

Return: $h^{FF}(s) = |\text{plan}|$

FF Properties

- **NOT admissible** (can overestimate due to greedy extraction)
- Very informed — works well for satisficing planning
- Complexity: $O(|A| \cdot |V|^2)$

D.32 Planning Complexity

Complexity Classes

Problem	Complexity
PlanEx (unbounded)	PSPACE-complete
PlanEx($\leq k$) bounded	NP-complete
PlanOptimal($\leq c$)	NP-complete

PART E: Abstractions & PDBs (L5)

E.33 Abstraction Fundamentals

Abstraction Mapping

An **abstraction** is a surjective function:

$$\alpha : S \to S^\alpha$$

Plain English: Group similar states together. Many concrete states → one abstract state.
If $\alpha(s_1) = \alpha(s_2)$, then s_1, s_2 are **equivalent** under α (we treat them as "the same")
Abstract state space: Θ^α with:

- $I^\alpha = \alpha(I)$ — abstract version of initial state
- $S_G^\alpha = \{\alpha(s) : s \in S_G\}$ — abstract goals
- $T^\alpha = \{(\alpha(s), a, \alpha(t)) : (s, a, t) \in T\}$ — abstract transitions

Abstraction Heuristic

$$h^\alpha(s) = h^*_{\Theta^\alpha}(\alpha(s))$$

Plain English: Solve the SIMPLER abstract problem exactly, use that cost as estimate.
Key properties:

- **Admissible:** $h^\alpha(s) \leq h^*(s)$ always (abstract paths can be shortcuts)
- **Consistent:** Triangle inequality holds
- Self-loops "don't count" — actions that do nothing in abstract space are free

Why it underestimates: Abstraction can "merge" states, creating shortcuts that don't exist in reality.

E.34 Pattern Database (PDB) Heuristics

Projection and PDB

Pattern: $P \subseteq V$ (subset of variables)
Projection: $\pi_P(s) = s|_P$ (restrict state to P)
Syntactic projection: $\Pi|_P$ = planning task with only P variables
PDB heuristic: $h^P(s) = h^*_{\Theta^P}(\pi_P(s))$

PDB Computation Steps

1. **Syntactic projection:** Build $\Pi|_P$
2. **Build abstract space:** Forward search from $I|_P$
3. **Backward Dijkstra:** Compute h^* from goal states
4. **Store PDB:** Hash table for $O(1)$ lookup

Hash function: index $= s(v_1) + |D_{v_1}| \cdot s(v_2) + \ldots$

E.35 Orthogonal Patterns (CRITICAL)

Affectedness and Orthogonality

Affected variables: $\text{aff}(a) = \{v \in V : \text{eff}_a(v) \neq \bot\}$
Plain English: Which variables does action a change?
Action affects pattern: a affects P iff $\text{aff}(a) \cap P \neq \emptyset$
Orthogonal patterns: $P_1 \perp P_2$ iff:

$$\forall a \in A : |\text{aff}(a) \cap (P_1 \cup P_2)| \leq 1$$

Plain English: No action touches BOTH patterns. Each action is "dedicated" to at most one pattern.

Critical Distinction

Disjoint $\not\Rightarrow$ Orthogonal!
Patterns may be disjoint ($P_1 \cap P_2 = \emptyset$) but NOT orthogonal if some action affects both.
Example: `fancyCar` affects both A and B $\Rightarrow$ $\{A\}$ and $\{B\}$ not orthogonal!

Additivity Theorem

If $P_1, P_2, \ldots, P_k$ are **pairwise orthogonal**:

$$h^{\{P_1,\ldots,P_k\}}(s) = \sum_{i=1}^{k} h^{P_i}(s) \quad \text{is } \textbf{admissible}$$

Non-orthogonal: Use max instead (always safe)

E.36 Cliques Heuristic

Orthogonality Graph and Cliques

Orthogonality graph:

- Vertices = patterns in collection $\mathcal{C}$
- Edge (P_i, P_j) iff $P_i \perp P_j$

Cliques heuristic:

$$h_{\mathcal{C}}^{\text{cliques}}(s) = \max_{D \in \text{cliques}(\mathcal{C})} \sum_{P \in D} h^P(s)$$

Sum within each clique, take max over all maximal cliques.

E.37 Pathological Abstractions

Three Extremes to Avoid

Type	**Compute α**	**Compute h^***	**Quality**
One-State	Trivial	Trivial	Useless ($h = 0$)
Identity	Trivial	NP-hard	Perfect
Perfect (h^*)	NP-hard	Easy	Perfect
Practical PDB	**Fast**	**Polynomial**	**Good**

PART F: Partial Order Reduction (L6)

F.38 Motivation: Why POR?

Even Almost-Perfect Heuristics Fail!

Problem: With $h(s) = h^*(s) - 1$, search still explodes exponentially!
Cause: Plan permutations — different orderings of same actions
Example: sock_L, sock_R can be done in any order. A* explores ALL orderings!
Solution: POR prunes equivalent permutations — works **orthogonally** to heuristics

F.39 Action Dependencies (CRITICAL)

Four Key Relationships

Relation	Definition	Plain English
Enabling	$a_2 \notin \mathcal{A}(s)$ but $a_2 \in \mathcal{A}(s[a_1])$	a_1 unlocks a_2
Disabling	$a_2 \in \mathcal{A}(s)$ but $a_2 \notin \mathcal{A}(s[a_1])$	a_1 locks a_2
Conflict	$s[\langle a_1, a_2\rangle] \neq s[\langle a_2, a_1\rangle]$	Order matters!
Interfering	Conflict OR Disabling	Can't safely reorder

Examples:
- **Enabling:** drive(A,B) enables drive(B,C) — must be at B first!
- **Disabling:** pickup(ball) disables another pickup(ball) — ball is gone
- **Conflict:** paint(red) vs paint(blue) — final color depends on order

Commutative Actions (KEY)

a_1, a_2 are **commutative** iff:
- NOT interfering (no conflict, no disabling)
- Neither enables the other

Commutative actions can be done in any order with same result!
POR goal: Prune orderings of commutative actions

F.40 Strong Stubborn Sets (S3)

Strong Stubborn Set Properties

A set $S \subseteq A$ is a **Strong Stubborn Set** for state s if:
1. **Goal-awareness:** If $G \not\subseteq s$, then $S \cap \mathcal{A}_{s \to^* G} \neq \emptyset$
2. **Interference closure:** If $a \in S \cap \mathcal{A}(s)$ and b interferes with a, then $b \in S$
3. **Enabling closure:** If $a \in S \setminus \mathcal{A}(s)$, then $S \cap \mathcal{A}_{s \to^* a} \neq \emptyset$

Pruning function: $\rho(s) = S \cap \mathcal{A}(s)$ (applicable actions from S3)

S3 Algorithm

$S \leftarrow \emptyset$ {Start with empty set}
if $G \not\subseteq s$ **then**
  {Goal not yet satisfied?}
  Pick $a \in \mathcal{A}_{s \to^* G}$; Add to S {Add action needed for goal}
end if
while closure not satisfied **do**
  {Keep adding until stable}
  if $\exists a \in S \setminus \mathcal{A}(s)$ without enabling action in S **then**
    Pick $a' \in \mathcal{A}_{s \to^* a}$; Add a' to S {Add action to ENABLE a}
  end if
  if $\exists a \in S \cap \mathcal{A}(s)$ with interfering $b \notin S$ **then**
    Add b to S {Add actions that INTERFERE}
  end if
end while
return S

Plain English:

1. Start with an action we need to reach the goal
2. For non-applicable actions in S: add something that can enable them
3. For applicable actions in S: add everything that interferes with them
4. Repeat until nothing new gets added
5. **Result:** Only expand actions in $S \cap \mathcal{A}(s)$ — safely prune the rest!

F.41 S3 Admissibility

Main Theorem

Theorem: Strong Stubborn Sets are **admissible**
$\Rightarrow$ A* with S3 pruning finds **optimal** solutions
Proof intuition:

- Goal-awareness ensures we can reach goal
- Enabling closure ensures we can execute needed actions
- Interference closure ensures we don't break dependencies

F.42 Quick Reference

Necessary Enabling Sets

$\mathcal{A}_{s \to^* a}$ = actions where at least one MUST be used before a
$\mathcal{A}_{s \to^* G}$ = actions where at least one MUST be used to reach goal
Example: To do drive(1,B,C), must first do drive(1,A,B) or teleport(1,B)

When POR Helps Most

- Many independent packages/objects
- Actions on different variables are commutative
- Gripper, Logistics, Barman domains

When POR helps less: Tightly coupled domains where most actions interfere